this

Decided by how a function is called, not where it is written

Call style	this is	Why
`new Fn()`	new object	constructor call
`fn.call(obj)`	obj	you choose it
`obj.fn()`	obj	left of the dot
`fn()`	undefined	strict mode
`fn()`	globalThis	sloppy mode
`addEventListener`	the element	regular fn only

1 call & apply

```
// both run right now
fn.call(obj, 1, 2)
fn.apply(obj, [1, 2])
// args: list vs array
```

2 bind

```
const g = fn.bind(obj);
new g();   // new beats bind
const f = obj.method;
f();       // this is lost
```

TRAP What does this log?

```
function who() { return this.name; }
const a = who.bind({ name: "A" });
console.log(a.call({ name: "B" }));
```

A

bind locks **this** for good. **call** can't change it.

KEY TAKEAWAY

- `call site` this = how it is called
- `bind` this locked, only new beats it
- `arrow` Outer this, never rebound

? Follow-up questions to expect

- **What decides the value of `this`?** → How the function is called.
- **Which `this` does an arrow function have?** → The outer one, even in object methods.
- **Can you rebind a bound function?** → call can't. new can: it ignores the bound this.
- **Why does `this` get lost in a callback?** → Called without its owner: use bind or an arrow.